○ Decision Making
● Information Sharing
○ Directions

dtDynamics 0.1.3 사용과 Legacy Migration

2026년 9월 16일

Joonhee Jo

Checked setup · Native API · 설치 경계

HYUNDAI

오늘의 흐름

01 현재 package 경계 — dynamics ingredient engine

02 내부 algorithm — MuJoCo-style baseline과 CCRBA

03 안전한 사용 — load, validate, checked compile, resize, cache, run

04 Migration — legacy engine에서 native API로

05 Validation과 성능 — independent reference와 runtime evidence

Part 1 — What dtDynamics is now

An ingredient engine, not a simulator

`dtDynamics` computes the kinematic, dynamic, centroidal, and derivative quantities that WBC, state estimation, and planners consume.

- deterministic memory ownership after initialization
- one fixed, documented coordinate convention
- `dtMath` as the **only** math dependency
- MuJoCo-compatible indexing and XML-oriented model semantics
- controller-ready quantities general simulators do not expose

Not in scope

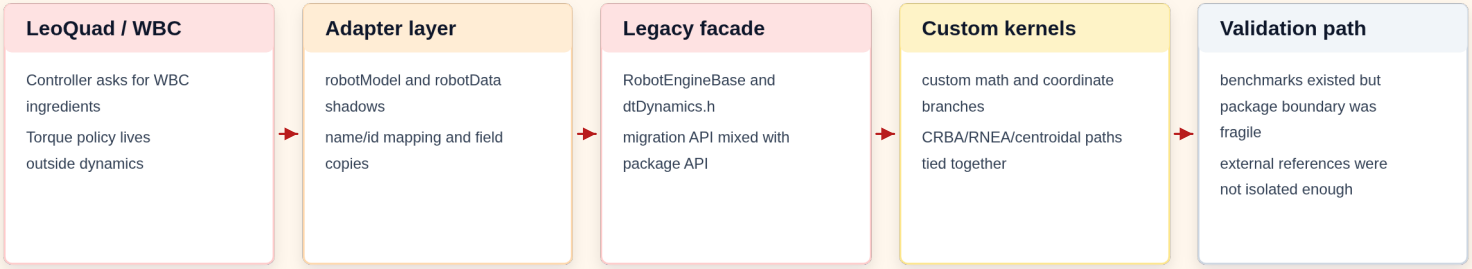
- contact solving
- forward simulation integrators
- **final torque command** — controllers own torque
- closed chains before the equality layer exists

As-Is: legacy integration path

As-Is: coupled legacy dynamics stack

Controller data, shadow model structures, legacy kernels, and validation concerns were mixed across package boundaries.

MAIN DATA FLOW



KEY INTERPRETATION

Package boundary risk

Dynamics, adapters, simulator-facing assumptions, and controller migration lived too close together.

Convention drift

Unsigned/sentinel id mismatch and multiple coordinate modes made MuJoCo-style indexing hard to guarantee.

Runtime friction

Copied data structures and broad recomputation made the control loop harder to reason about and profile.

Release fragility

Install-prefix and dependency layout were easy to misconfigure for downstream projects.

Redesign target

Keep and add the useful physical quantities, but move to a standalone native package with MuJoCo-style model/data conventions, dtMath storage, direct controller access, and separate validation tools.

To-Be: standalone native package

To-Be: standalone dt::Dynamics package

Native dtDyn* API, dtMath-first storage, MuJoCo-style indexing, direct WBC data access, and isolated reference validation.

MAIN DATA FLOW

dtDynSpec

- MJCF-like XML syntax
- global body and fixed bodies before compile

→

dtDynCompile

- builds deterministic arrays
- resolves names, parents, inertias, and dof addresses

→

dtDynModel

- MuJoCo-style signed ids
- qpos/qvel/body/site layouts match native conventions

→

dtDynData

- single runtime output store
- M, C, g, bias, Jacobians, CoM, centroidal terms

→

LeoQuad / WBC

- consumes dtDynData directly
- controller computes tau from selected ingredients

KEY INTERPRETATION

Production boundary

dtDynamics links as dt::Dynamics and depends on dtMath only. No MuJoCo or Pinocchio in production.

Dynamics pipeline

ForwardKinematics, ComputeDynamicsCCRBA/RNEA, ComputeJacobians, and term helpers are explicit.

Validation sandbox

MuJoCo 3.8.0 and Pinocchio 3.9.0 stay under package validation paths and benchmarks.

Install layout

Installed directly to ARTF: include/dtDynamics, lib/libdtDynamics.a, lib/cmake/dtDynamics.

Release target

Release candidate: package version 0.1.3.

CMake package boundary

Production consumers link the installed package target:

```
find_package(dtDynamics 0.1.3  EXACT CONFIG REQUIRED)
target_link_libraries(my_controller PRIVATE dt::Dynamics)
```

- production target is **dtMath-only**
- MuJoCo / Pinocchio are linked only by dedicated validation and benchmark executables
- LeoQuad prepends `DTDYNAMICS_INSTALL_PREFIX` to `CMAKE_PREFIX_PATH`

Include-order rule

If a project also includes a shared ART Framework include directory, the installed `dtDynamics` include path must come **before** any older `ART_Framework/include/dtDynamics` copy.

Mixing a new `libdtDynamics.a` with stale headers changes object layout and can corrupt `dtDynModel` / `dtDynData` arrays at runtime.

Part 2 — Algorithms inside

MuJoCo-style smooth-dynamics baseline

Native `dtMath` reimplementation of the MuJoCo 3.8.0 stage structure:

1. body kinematics — `mj_kinematics`
2. inertial-frame / subtree-CoM mapping — `mj_comPos`
3. mass matrix: CRBA then armature — `mj_makeM` / `mj_crb`
4. optional factorization — `mj_factorM`
5. CoM-frame velocity, motion-subspace derivatives — `mj_comVel`
6. RNEA bias forces — `mj_rne` (gravity, coriolis, nonlinear)
7. body/site Jacobians — `mj_jac*`
8. Jacobian time variation — `mj_jacDot`

> MuJoCo names are **structural references only** — the package neither includes nor links MuJoCo.

CCRBA: the dtDynamics-specific layer

Control-oriented Composite Rigid Body Algorithm — adds what MuJoCo does not expose in controller form:

- controller-compatible `Cmat` where `coriolisTorq = Cmat * qvel`
- CMM `cmAmat` and derivative `cmAmatDot`
- CoM Jacobian `comJmat` and `comJmatDot`
- centroidal momentum `centMom`
- `MmatDot` for derivative-aware control

One-pass design

`coriolisTorq` is accumulated **while `Cmat` is filled** — no second dense `Cmat * qvel` pass.

Gravity, Coriolis, and nonlinear torques come from the CCRBA pass itself instead of a separate RNEA call.

Two explicit entry points, no generic alias

ComputeDynamicsRNEA

Lighter path: MuJoCo-step-like base ingredients only.

```
// Mmat, gravityTorq, coriolisTorq, nonlinTorq
ComputeDynamicsRNEA(model, data, gravity,
                    DTDYN_RNEA_TERM_COM_POS);
```

Optional flags: `DTDYN_RNEA_TERM_COM_POS` only.

ComputeDynamicsCCRBA

Base ingredients plus `Cmat`, plus opt-in centroidal terms.

```
ComputeDynamicsCCRBA(model, data, gravity,
    DTDYN_CCRBA_TERM_COM_POS |
    DTDYN_CCRBA_TERM_CMAMAT  |
    DTDYN_CCRBA_TERM_CENT_MOM);
```

Flags: `CMAMAT`, `CMAMAT_DOT`, `COM_JMAT`, `COM_JMAT_DOT`, `CENT_MOM`, `COM_VEL`, `MMAT_DOT`.

Neither call computes body/site Jacobians — request them separately with `ComputeJacobians`.

World-body and fixed-body safety rules

The recursion is hardened against the classic CCRBA failure mode: treating world body `0` as a physical body corrupts centroidal terms.

World body `0`

- work buffers explicitly zeroed
- physical body loops start at index `1`
- backward accumulation stops before `0`
- never owns DOF, mass, or inertia

Fixed body (no joint)

- physical: mass, inertia, sites, children
- contributes through parent composite inertia
- creates **no** columns in `M`, `C`, Jacobians, CMM
- body-indexed arrays sized by `nbody`, DOF-indexed by `nv`

Multi-root models: centroidal outputs are stacked **per root** (`6*nroot` rows), driven by `body_rootid` metadata.

Part 3 — How to use dtDynamics

Build a model from XML


```
#include <dtDynamics/dtDynCompile.h>
#include <dtDynamics/dtDynData.h>
#include <dtDynamics/dtDynSpec.h>

using namespace dt::Dynamics;

const dtDynSpec spec =
    dtDynSpec::LoadXmlFile(
        "model/LeoQuad_QD1.xml");

// Optional normalized native-spec export.
spec.SaveXmlFile(
    "model/LeoQuad_QD1.native.xml");

const dtDynModel model =
    CompileSpecChecked(spec);
dtDynData data;
data.Resize(model);
```

Build a programmatic spec

```
dtDynSpec spec;
const dtDynIndex base =
    spec.AddBody("base", 0);
spec.AddJoint(
    base, "floating_base",
    dtDynJointType::Free);

const dtDynIndex link =
    spec.AddBody("link", base);
spec.EditBody(link, [](dtDynBodySpec &draft) {
    draft.pos(0) = 0.35;
    draft.inertial.mass = 1.0;
});

const dtDynIndex hip = spec.AddJoint(
    link, "hip_pitch",
    dtDynJointType::Hinge);
spec.EditJoint(hip, [](dtDynJointSpec &draft) {
    draft.axis(1) = 1.0;
});
const dtDynIndex foot =
    spec.AddSite(link, "foot");
spec.EditSite(foot, [](dtDynSiteSpec &draft) {
    draft.pos(2) = -0.25;
});
```

Compile editable/external specs through `CompileSpecChecked`; reserve raw `CompileSpec` for separately validated trusted setup data.

Set state and compute

```
// qpos: base pos, quaternion w x y z, joints
data.qpos(2) = 0.5;
data.qpos(3) = 1.0;   // quaternion w
data.qpos(7) = 0.2;   // first hinge

// qvel: baseLinVel_G, baseAngVel_B, joints
data.qvel.SetZero();
data.qvel(0) = 0.1;
data.qvel(4) = 0.2;
data.qvel(6) = 0.3;

dt::Math::Vector3<dtDynReal> gravity;
gravity(2) = -9.81;
```

```
ForwardKinematics(model, data);

constexpr dtDynCcrbaTermFlags wbcTerms =
    DTDYN_CCRBA_TERM_COM_POS |
    DTDYN_CCRBA_TERM_CMAMAT |
    DTDYN_CCRBA_TERM_CENT_MOM;

ComputeDynamicsCCRBA(model, data, gravity, wbcTerms);
```

Request only the optional terms the controller needs — heavier centroidal outputs cost time every tick.

Jacobians: indexed for realtime, terms for one-offs

Realtime loop — cache IDs once, pass them every tick:

```
dtDynArray<dtDynIndex> bodyIds;
dtDynArray<dtDynIndex> siteIds;
const auto footSite = model.SiteId("foot");
if (footSite == DTDYN_INVALID_INDEX)
    throw std::runtime_error("required foot site missing");
bodyIds.push_back(model.root_bodyid[0]);
siteIds.push_back(footSite);

ComputeJacobians(model, data, bodyIds, siteIds);
```

Include the root/base body when the controller consumes a base Jacobian — site-only is not the LeoQuad contract.

One-off task point — `J`, `Jdot`, `Jdot*qvel` in one call. Reuse the checked `footSite` cached during setup:

```
ComputeSiteJacobianTerms(model, data,
                         footSite,
                         &jacp, &jacr,
                         &jacpDot, &jacrDot,
                         &linJdotQvel,
                         &angJdotQvel);
```

Pass `nullptr` for outputs you do not need; shared column terms are computed once.

Output quantity reference (core set)

Field	Shape	Prerequisite	Controller use
`xpos`, `xmat`, `xipos`	per body	`ForwardKinematics` or dynamics call	body pose / CoM export
`site_xpos`, `site_xmat`	per site	`ForwardKinematics` or dynamics call	task pose export
`Mmat`	`nv x nv`	RNEA or CCRBA	WBC inertia matrix
`Cmat`	`nv x nv`	CCRBA	explicit Coriolis matrix
`gravityTorq`, `coriolisTorq`, `nonlinTorq`	`nv`	RNEA or CCRBA	bias-force ingredients
`body_jacp/r`, `site_jacp/r` (+ `_dot`)	`3 x nv`	`ComputeJacobians` / `*Terms`	task constraints, `Jdot*qvel`
`cmAmat`, `cmAmatDot`	`6*nr x nv`	CCRBA + `CMAMAT(_DOT)`	centroidal momentum control
`centMom`	`6*nr`	CCRBA + `CENT_MOM`	momentum feedback
`comPos`, `comVel`, `comJmat(Dot)`	`3*nr` rows	CCRBA + `COM_*` flags	CoM tracking
`MmatDot`	`nv x nv`	CCRBA + `MMAT_DOT`	derivative-aware control

`nr = max(nroot, 1)`; centroidal blocks are stacked per root, rows ordered `[linear; angular]`.

The LeoQuad realtime sequence

This is the migrated production pattern in `Robot::UpdateDynamics`:

```
// 1. stage state (qpos/qvel convention: basePos_G, quat, baseLinVel_G, baseAngVel_B, joints)
data.dynData.qpos(...) = ...;
data.dynData.qvel(...) = ...;

// 2. reduced CCRBA — only the WBC terms
constexpr dt::Dynamics::dtDynCcrbaTermFlags dynamicsTerms =
    dt::Dynamics::DTDYN_CCRBA_TERM_COM_POS |
    dt::Dynamics::DTDYN_CCRBA_TERM_CMAMAT |
    dt::Dynamics::DTDYN_CCRBA_TERM_CENT_MOM;
dt::Dynamics::ComputeDynamicsCCRBA(model.dynModel, data.dynData, gravity, dynamicsTerms);

// 3. indexed Jacobians — base body + task sites cached at init
dt::Dynamics::ComputeJacobians(model.dynModel, data.dynData,
                       model.dynJacobianBodyIds, model.dynJacobianSiteIds);

// 4. export: read dtDynData fields directly into controller-facing terms
```

No XML parsing or name lookup inside the tick. On the documented pre-sized benchmark path, the C++ `new` counter reports zero calls; this is not a whole-process heap guarantee.

Part 4 — Migrating from legacy dtDynamics

Legacy vs redesigned, at a glance

Aspect	Legacy (`leoquad-control_legacy`)	Redesigned (current)
Engine type	`RobotEngineBase<DOF_TOTAL, NUM_BODY, NUM_JOINT, COORD, T>` template	`dt::Dynamics` namespace API, runtime-sized
Model loading	`mj_loadXML` + `mj_makeData` — **MuJoCo in production**	`LoadXmlFile` / `DecodeXml` → `CompileSpecChecked` → `Resize` — dtMath-only
Sizes	compile-time template parameters	compiled `dtDynModel` dimensions (`nq` , `nv` , …)
Body indices	manual re-basing: `bodyId[i] - startBodyId`	MuJoCo-compatible IDs + `BodyId` / `SiteId` lookup
Velocity basis	`COORDINATE_TYPE` selector 0/1/2	one convention: `[linVel_G, angVel_B, qdot]`
Outputs	`Get*` calls copy into `RobotData` shadow fields	read `dtDynData` fields directly
Jacobians	per-body `GetBodyJacob` inside update loop	one indexed `ComputeJacobians` call

Step 1 — Model loading

Legacy

```
// robot.h
dtDyn::RobotEngineBase<DOF_TOTAL, NUM_BODY,
    NUM_JOINT, 0, SYSREAL> physicsEngine;

// InitModel: MuJoCo loads the model in production
reVar.m_modelMUJOCO =
    mj_loadXML(modelFile, nullptr, error, 1000);
reVar.m_dataMUJOCO =
    mj_makeData(reVar.m_modelMUJOCO);

startBodyId = mj_name2id(m, mjOBJ_BODY,
                         robotBodyNames[0].c_str());
model.bodyId[i] = bodyId[i] - startBodyId;
```

Redesigned

```
// robotModel.h / robotData.h
dt::Dynamics::dtDynModel dynModel;
dt::Dynamics::dtDynData dynData;

// InitializeDynamicsModel: native decode/compile
const dtDynSpec spec =
    dtDynSpec::LoadXmlFile(fileName);
model.dynModel = dt::Dynamics::CompileSpecChecked(spec);
data.dynData.Resize(model.dynModel);

if (model.dynModel.nq != DOF_TOTAL_QUAT ||
    model.dynModel.nv != DOF_TOTAL)
    throw std::runtime_error("dimension mismatch");
```

Step 2 — Cache IDs by name, once

Legacy

Local body numbering, re-derived from MuJoCo each init:

```
data.actTaskStates[t].id =
    m->site_bodyid[mj_name2id(m, mjOBJ_SITE,
                              taskName)]
    - startBodyId;
```

Controllers then indexed everything with the shifted local IDs.

Redesigned

Stable name lookup on the compiled model:

```
const auto siteId = model.dynModel.SiteId(
    robotTaskNames[t]);
if (siteId == dt::Dynamics::DTDYN_INVALID_INDEX) {
    throw std::runtime_error("required task missing");
}
model.dynTaskSiteIds[t] = siteId;
model.dynTaskBodyIds[t] =
    model.dynModel.site_bodyid[siteId];

model.dynJacobianBodyIds.push_back(
    model.dynBodyIds[0]);
model.dynJacobianSiteIds.push_back(siteId);
```

Validate every ID at init; never look up names in the tick.

Step 3 — Per-tick update

Legacy

```
physicsEngine.ComputeKinematics(model, data);
physicsEngine.ComputeDynamics(model, data);

physicsEngine.GetBase2SubtreeCom(model, data,
    model.bodyId[0], comPosB_G);
physicsEngine.GetComVel(model, data, data.comVel);

for (i = 0; i < NUM_TASK; i++) {
    physicsEngine.GetBase2BodyPose(...);
    physicsEngine.GetBodyJacob(...);
    physicsEngine.GetBodyJacobDeriv(...);
}
physicsEngine.ComputeCentMomentumDot(model, data);
data.centMom = data.cmAmat * data.qVel;
```

Redesigned

```
dt::Dynamics::ComputeDynamicsCCRBA(
    model.dynModel, data.dynData, gravity,
    DTDYN_CCRBA_TERM_COM_POS |
    DTDYN_CCRBA_TERM_CMAMAT |
    DTDYN_CCRBA_TERM_CENT_MOM);

dt::Dynamics::ComputeJacobians(
    model.dynModel, data.dynData,
    model.dynJacobianBodyIds,
    model.dynJacobianSiteIds);

// poses, Jacobians, comPos, centMom:
// read them from data.dynData fields
```

Call-site mapping

Legacy call	Redesigned equivalent
`InitModel(model, data, file)`	`LoadXmlFile` → `CompileSpecChecked` → `dtDynData::Resize`
`ComputeKinematics`	`ForwardKinematics` (also run inside dynamics calls)
`ComputeDynamics`	`ComputeDynamicsRNEA` or `ComputeDynamicsCCRBA`
`GetBodyJacob`, `GetBodyLinJacob`	`ComputeJacobians` → `body_jacp` / `body_jacr`
`GetBodyJacobDeriv`	same call → `body_jacp_dot` / `body_jacr_dot`, or `Compute*JacobianTerms`
`GetBase2SubtreeCom`	`subtree_com` / `comPos` via `COM_POS` flag
`GetComVel`	`comVel` via `COM_VEL` flag
`GetCMMat_I`	`cmAmat`, `comJmat`, `gravityTorq` via `CMAMAT` / `COM_JMAT` flags
`GetCMMatDeriv_I`, `ComputeCentMomentumDot`	`cmAmatDot`, `comJmatDot` via `*_DOT` flags
`data.centMom = data.cmAmat * data.qVel`	`centMom` via `CENT_MOM` flag
`GetInertia_I`, `GetCoriolis_I` (frame transforms)	removed — all outputs already in the single convention

Migration pitfalls

Pitfall	Correct approach
Assuming a `COORDINATE_TYPE` selector still exists	Only `[baseLinVel_G, baseAngVel_B, qdot]` is supported
Reusing legacy re-based local indices	Cache checked `BodyId` / `JointId` / `SiteId` / `ActuatorId` results at init
Expecting the dynamics call to fill Jacobians	Call `ComputeJacobians` separately (indexed for realtime)
Copying all of `dtDynData` into shadow structs	Read `dtDynData` fields directly; export only what control needs
Linking MuJoCo in production for model loading	XML decode is native; MuJoCo is validation-only
Stale `ART_Framework/include/dtDynamics` headers first	Installed package include dir must precede the shared prefix

Part 5 — Validation and performance

External references, outside the package

MuJoCo 3.8.0

- XML semantics and indexing
- transforms, subtree CoM
- `mj_makeM` mass matrix, bias vector
- task Jacobians, `Jdot*qdot`

Pinocchio 3.9.0

- CRBA, centroidal matrix and momentum
- CMM time derivative, CoM Jacobian
- Jacobian time variation

Fixture matrix

- minimal world/fixed-body model
- single and dual fixed-base manipulators
- fixed-base and floating-base multi-robot models
- floating-base quadruped
- four-bar fixture: rejection diagnostics until equality support

Both references live in a private `.validation_deps/` cache — never in the production package or the shared ART prefix.

LeoQuad runtime gates

Standalone package gates:

```
tools/validate_install_export.sh
tools/validate_package.sh
tools/audit_release_readiness.sh
```

External `leoquad-control` integration checkout (not shipped here):

```
cmake --build build --target simRobot
tools/validate_simrobot_trot.sh --smoke
tools/validate_simrobot_trot.sh --headless-trot
SIMROBOT_PROFILE_DYNAMICS=1 \
  tools/validate_external_dtdynamics_package.sh
```

What the gates check

- `--smoke` catches startup regressions such as unallocated dynamic vectors
- `--headless-trot` drives the scripted `i` → `f` → `i` trot sequence without GLFW and checks the controller reaches `CTRL_GAIT + GAIT_TROT`
- the external gate rebuilds the package, runs MuJoCo/Pinocchio tests, installs, and re-runs the trot gate against the installed package

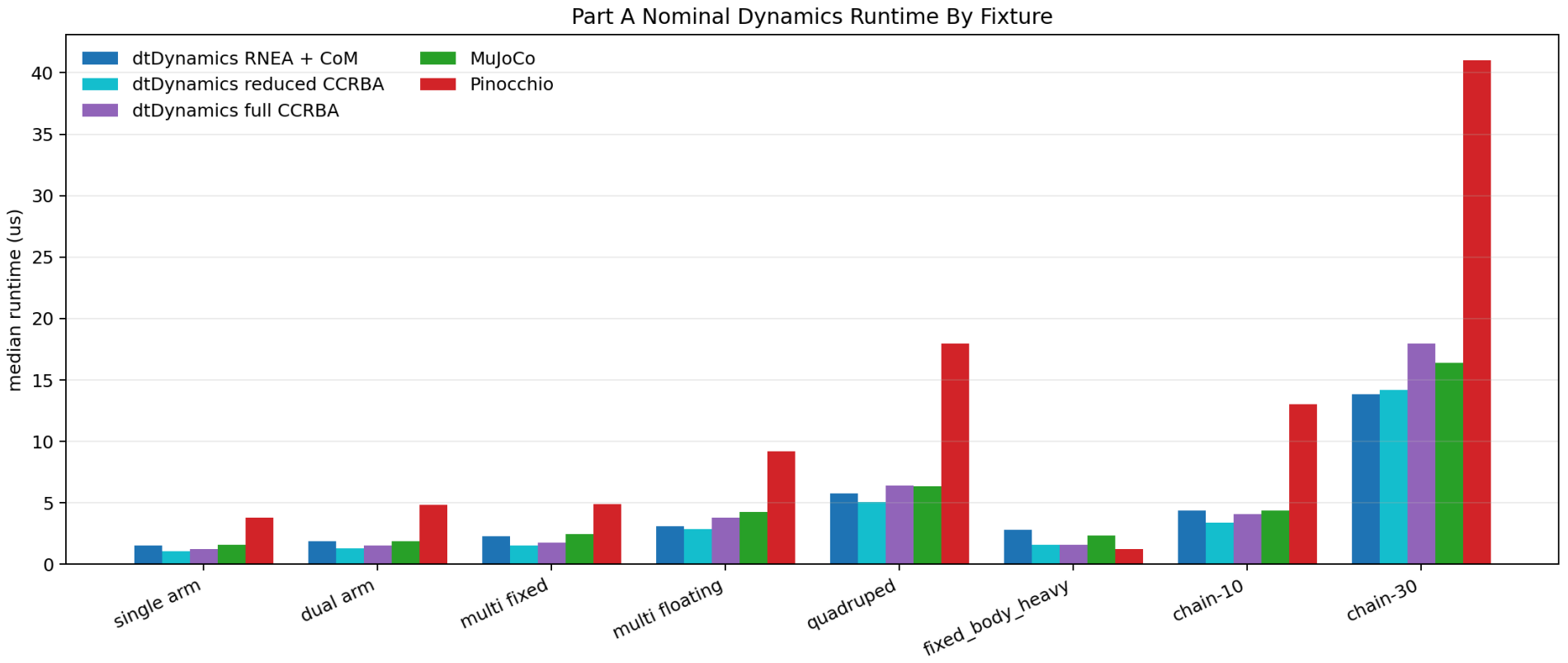
Canonical Part A quadruped snapshot

Each cell is `median / p95` in microseconds for `quadruped_floating_base`.

State	RNEA + CoM	Reduced CCRBA	Full CCRBA	MuJoCo	Pinocchio
nominal	5.751 / 5.881	5.073 / 5.213	6.398 / 6.560	6.320 / 6.384	17.962 / 18.090
asymmetric	5.877 / 6.032	5.182 / 5.310	6.518 / 6.685	7.000 / 7.072	18.367 / 18.511
nonzero_velocity	6.049 / 6.187	6.907 / 7.045	8.166 / 8.293	7.039 / 7.119	18.364 / 18.507

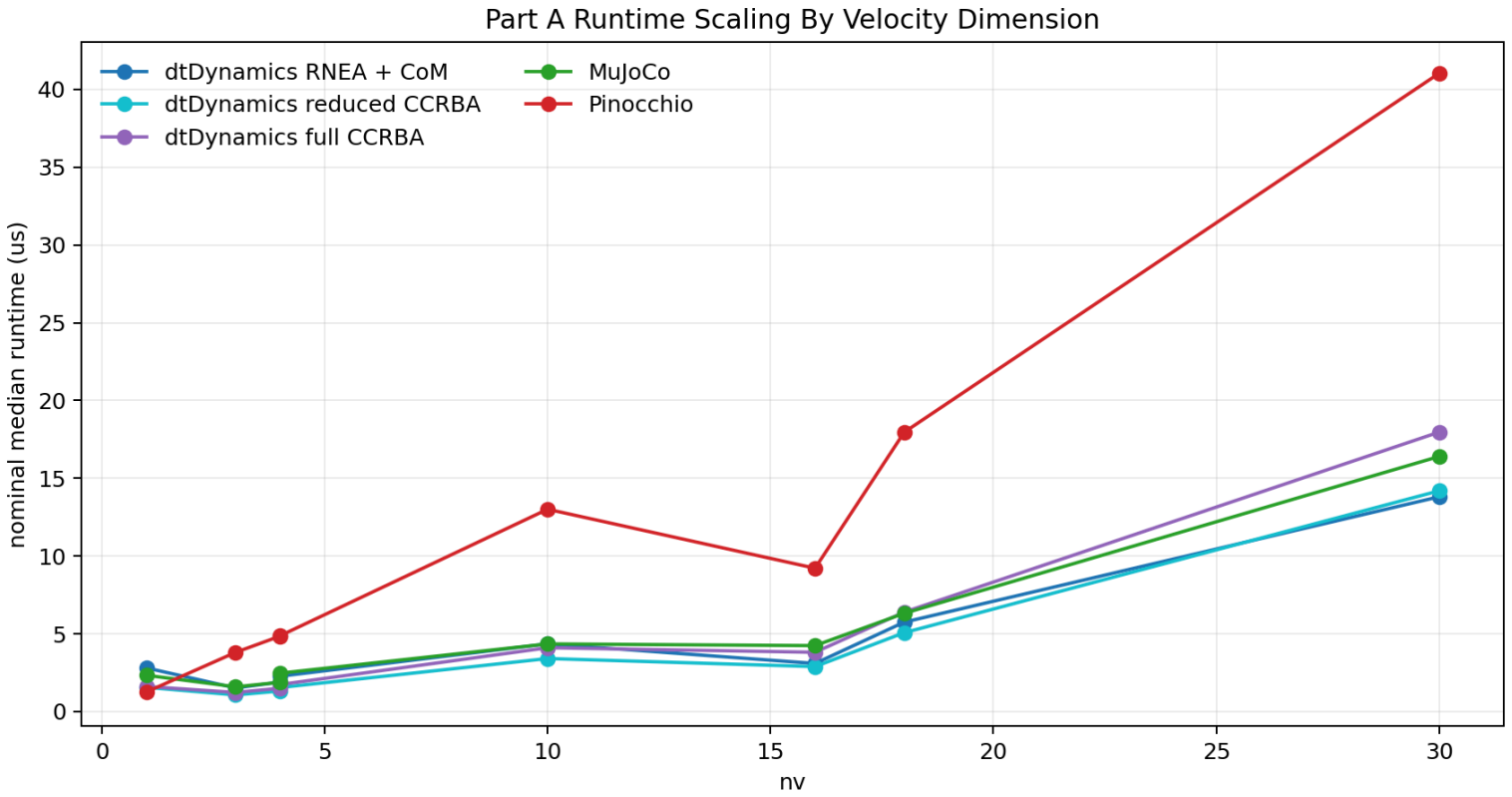
- Release, CPU 0 affinity, seven batches, 20 warmup + 200 measured iterations per batch.
- Values are medians of batch-level summaries, not pooled percentiles.
- MuJoCo and Pinocchio are contextual reference-call timings; output contracts are not identical to native CCRBA.

Nominal runtime across fixtures



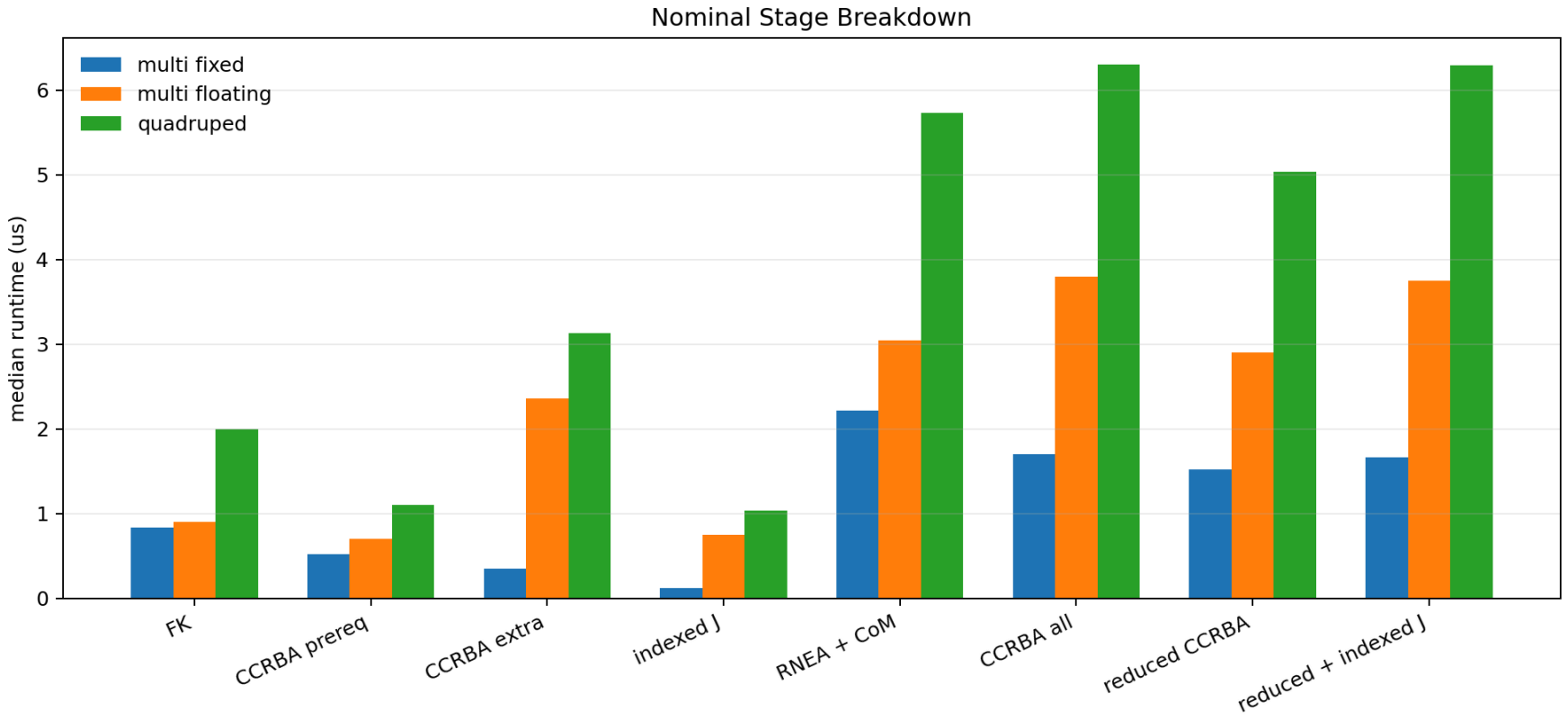
The figure shows nominal medians only. Native calls request different output sets; MuJoCo and Pinocchio remain contextual because their output contracts differ.

Runtime scaling by velocity dimension



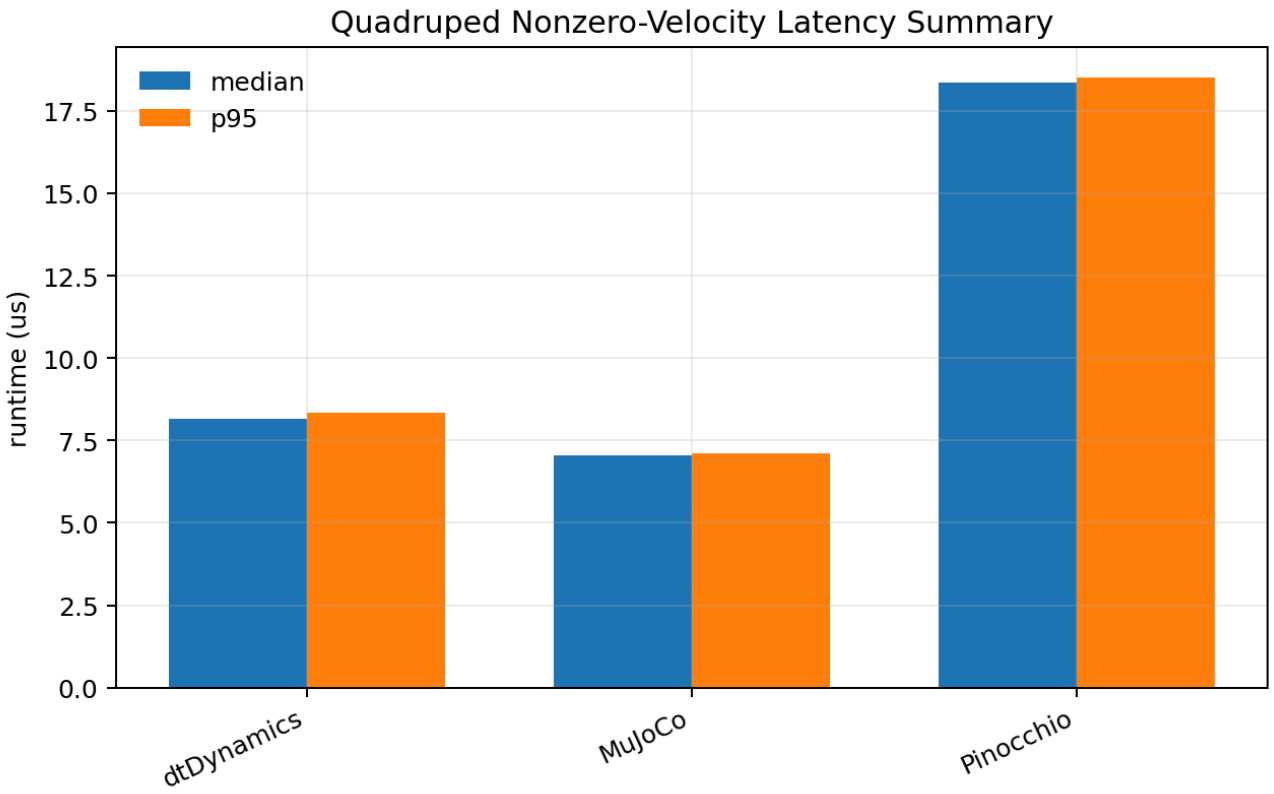
Each marker is a different fixture; connected lines are visual guides, not fitted complexity curves. Equal `nv` does not imply equal topology or requested work.

Independently timed stage breakdown



The stage rows are independently prepared and non-additive; do not sum bars to reconstruct another public-call latency.

Nonzero-velocity reference latency



This is the reference call sequence, not the three native Part A scopes. Bars are batch-summary median/p99 evidence, not hard-realtime guarantees.

Canonical LeoQuad real-XML snapshot

All cells are central medians in microseconds for the current `LeoQuad_QD1.xml`.

State	CCRBA all	RNEA + CoM	Reduced CCRBA	Reduced + indexed J
nominal	6.383	5.845	5.162	7.501
asymmetric	6.529	5.922	5.255	7.630
nonzero_velocity	8.240	6.038	6.917	9.213

- Same Release, CPU-affined seven-batch protocol as Part A.
- Reduced CCRBA requests `COM_POS | CMAMAT | CENT_MOM`; the controller-shaped row then adds one root-body and four foot-site indexed Jacobians.
- The external XML remains opt-in and is not an installed-package dependency.

Setup and allocation boundaries

5.263 / 11.240 µs
quadruped compile median / p99

4.249 / 4.983 µs
fresh data-resize median / p99

31 calls
31 counted C++ allocation calls per fresh resize iteration

- Compile and resize are setup-only rows outside the realtime loop.
- Measured native dynamics and stage rows reported zero counted C++ `operator new` allocations.
- The allocation counter does not cover direct `malloc` / `calloc` / `realloc` paths or certify whole-process heap freedom.
- `docs/benchmark-results.md` is authoritative when these snapshot values change.

One-slide team explanation

1. `dtDynamics` is a dtMath-only ingredient engine: it computes `M`, `C`, gravity, Jacobians, and centroidal terms; controllers compute torque.
2. The baseline mirrors MuJoCo 3.8.0 smooth-dynamics stages; the CCRBA layer adds `Cmat`, CMM, and derivative terms in controller form.
3. Use it as: load/validate/checked-compile/resize once → check and cache IDs → stage `qpos/qvel` → selected dynamics/Jacobians → read `dtDynData`.
4. Migration from legacy: drop the template engine, MuJoCo production loading, `COORDINATE_TYPE`, local body IDs, and `Get*` shadow copies.
5. Every change is gated by MuJoCo/Pinocchio reference tests plus the LeoQuad headless trot validation.

> **Rule of thumb** — one convention, explicit entry points, cached IDs, no parsing in the tick, and pre-sized storage for the benchmarked zero-counted-`new` path.

Thank you

LeoQuad Control & Dynamics · dtDynamics usage & migration briefing